

hwde

A Claude Code skill that turns a written brief for a circuit board into the files a factory needs, checked at every step by the tools an engineer would use.

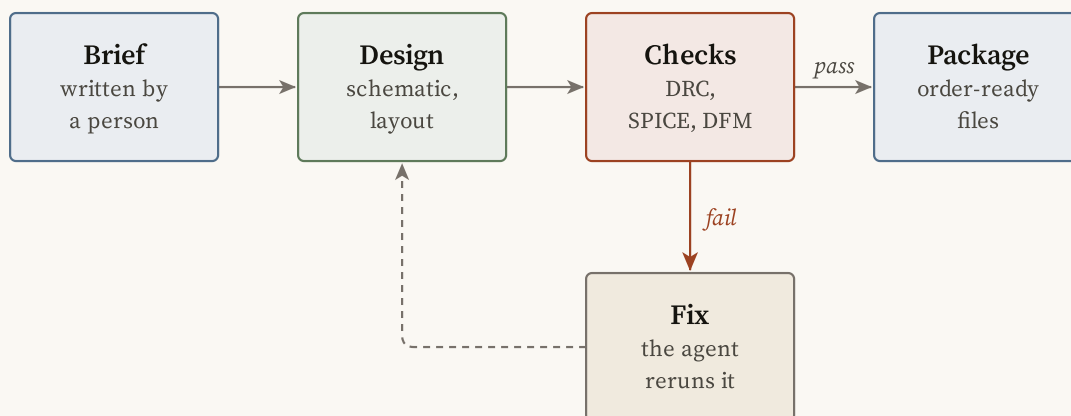
hwde is a router in front of about two dozen named subagents and a larger set of deterministic Python scripts. The subagents do the judgment calls: read a datasheet, choose a topology, place a part, decide whether a violation is real. The scripts do everything that can be checked instead of judged: a design-rule check, a SPICE simulation, whether a trace carries enough current. The split matters because a subagent can be wrong in a way that sounds confident; a script either passes or it doesn't.

The problem it answers is that a board has ten phases between a brief and an order, and most of what goes wrong in one phase only shows up as a failure two phases later. hwde keeps every phase's state in one file per board workspace, so a check can run again from scratch instead of trusting what an earlier phase said about itself.

How one board is built

1. **Intake and research.** A requirements analyst turns the brief into requirements; research subagents look up parts and reference designs; an architect commits to a topology. Human checkpoint 1 sits here.
2. **Parts and schematic.** Part-sourcer and librarian pull footprints; schematic-block agents draw it and a schematic-reviewer checks it; `check_current.py` and `check_decoupling.py` run alongside the ERC. Checkpoint 2.
3. **Placement and routing.** `place_seed.py` then `place_anneal.py` place parts, scoring the layout partly on how likely it is to route; `route_auto.py` drives Freerouting to route it, then `stitch_vias.py` and `plane_repair.py` clean up the copper. Checkpoint 3.
4. **Verification and DFM.** `sim_run.py` drives ngspice, `check_return_path.py` and `kicad-cli`'s DRC run against the routed board, a verify-reviewer reads the results, `dfm_check.py` runs the manufacturer's rule set. Checkpoint 4.
5. **Ordering.** `order_submit.py` packages gerbers, BOM and placement files and can call the fab's API. Checkpoint 5 is the only one that is never delegated: paying needs a person to type back the board name, quantity and total.

FIGURE 1 *one board*



A written brief becomes an order-ready board, checked at every step.

The hard parts

A gate failure has to turn into work, not a retry

A failed DRC or ERC run can list dozens of violations. `fix_dispatch.py` clusters them by net and region so one fixer agent gets one coherent work order instead of twenty agents each fixing one line. Fixing a placement is declared as an edit class, which marks every artifact downstream of it stale, so a route computed against the old placement can't be checked and passed by accident. The gate reruns after the fix; a board that fails the same gate twice escalates to a person instead of looping again.

Placement scored on a router it hasn't run yet

`place_anneal.py` moves parts to shrink an annealed cost function, but a placement that looks compact can still be unroutable. Its `--route-feedback` option scores candidate placements partly on predicted routability before Freerouting ever runs, so the annealer is nudged away from layouts that would only fail two steps later. Freerouting itself runs headless through a Specctra export and import, which on Linux needs a virtual display even though nothing is ever shown on one.

Checking the checkers

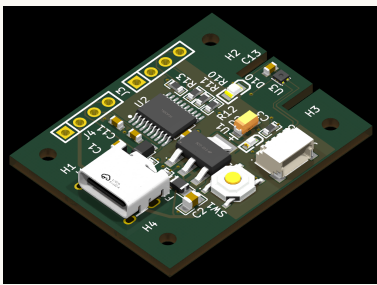
A checker that never fires would still show a green board. `tests/golden/manifest.yaml` keeps a set of golden boards, three clean and seven with one fault planted each: silk over a pad, a part rotated by the wrong footprint convention, an undersized power trace, skewed differential pairs, a missing return via, a plane split under a clock trace, a decoupling capacitor moved too far from its pin. Each fault names the exact checker that must catch it and where. It hasn't yet been measured how many of these plain KiCad's own DRC would have missed on its own.

Approval that isn't out of the agent's reach

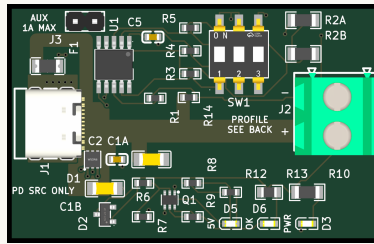
Reading `scripts/state.py`, the call that records a human checkpoint verdict, `record_human`, takes whatever checkpoint number and status it's given; nothing in the script checks who is calling it. Nothing has been seen approving its own gate in a run, but the code as written would let an agent call `state.py human --status approved` on itself. The fix is to move that call somewhere an agent's own commands can't reach, not to trust that it hasn't happened.

The one gate that is never delegated

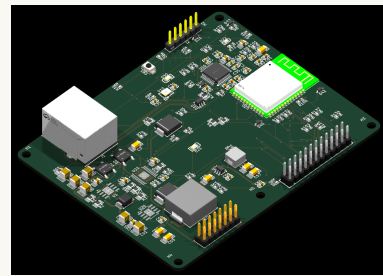
`order_submit.py` is the only script that spends money, so it refuses more than it proceeds: it will not touch a 4-plus-layer board (the fab's own API doesn't take one), refuses if an order record already exists for the board, refuses after any earlier attempt came back ambiguous, and needs a quote fetched fresh, a design hash matching what's on disk, and a typed confirmation naming the board, the quantity and the total before it calls out. The g0-sense run went from brief to an order-ready board in 13.6 hours with nobody watching and stopped right there, at the line only a person can cross.



g0-sense, order-ready in 13.6 h,
not ordered



pd-trigger, made, not powered
on



lumina-carrier, 4 layers, made,
not powered on

KiCad renders of three boards hwde designed.

What it doesn't do

- No electromagnetic field solver, so anything above a few GHz is out of scope.
- No public DFM API from the factory, so the factory's own manufacturability review still needs a person to read it.
- A green gate proves only the checks that ran. That's meant to be visible, not hidden behind a passing status.
- Two boards were made from hwde's output; neither has been powered on yet, so nothing here says a board actually works. `pd-trigger` was ordered through the fab's API; a separate 4-layer board went through the web cart by hand, because the API refuses that layer count.

From the public repo <https://github.com/ihsan-sa/hwde>: `SKILL.md`, `agents/*.md`, `scripts/state.py`, `scripts/order_submit.py`, `scripts/fix_dispatch.py`, `scripts/place_anneal.py` and `tests/golden/manifest.yaml`, read 26 September 2026.